

Operating System

LAB FILE



Submitted By:

SRISHTI

2024UCA1923

Computer Science and Engineering (Artificial Intelligence)

Index

S. No.	Name of the Experiment	Page
1.	Hands on Shell Commands	
2.	Process creation and termination for the operating system (fork, wait, signal, exit etc.).	
3.	Thread programming.	
4.	CPU scheduling algorithms: FCFS, SJF, Round Robin, Preemptive Priority Scheduling.	
5.	Inter process communication.	
6.	Critical Section problem.	
7.	Producer – Consumer problem using bounded and unbounded buffer.	
8.	Reader Writers problem, Dining Philosophers problem using semaphores.	
9.	Banker's algorithm.	
10.	Deadlock detection algorithm	
11.	Page replacement algorithms: LRU, LRU-Approximation, FIFO, Optimal.	
12.	File operation system calls (open, read, close, append etc.)	
13.	Disk scheduling algorithms: FCFS, SSTF, SCAN, CSCAN, LOOK, CLOOK.	

Lab Task 1: Hands on shell command

1. **pwd**: Present Working Directory

- Tells which directory we are working in

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ pwd  
/workspaces/OperatingSystem_Sem3  
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

2. **clear**

- Clears the terminal window

```
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

3. **date**

- To display the current date and time

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ date  
Sat Oct 25 12:30:26 UTC 2025  
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

4. **date +%D**

- Displays only the present date

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ date +%D  
10/25/25  
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

5. **date +%T**

- Displays only the present time

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ date +%T  
12:33:17  
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

6. date +%H:%M

- Displays current time in form of hours and minutes

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ date +%H:%M
12:34
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

7. ls -ltr

- List all files in long format, sorted by modification time, with the oldest files first

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ls -ltr
total 108
-rw-rw-rw- 1 codespace root      46569 Oct  8 06:28 a.exe
-rw-rw-rw- 1 codespace root       2528 Oct  8 06:28 ReadMe.md
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:10 Thread_Programming
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:25 Critical_Section_Problem
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:26 ProducerConsumer_Problems
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:26 'ReaderWriter_Problem & DiningPhilosopher
(Semaphores)'
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:29 BankersAlgorithm
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:30 DeadlockDetectionAlgorithm
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 10:21 CPU_SchedulingAlgorithms
-rwxrwxrwx 1 codespace codespace 24272 Oct 22 10:28 a.out
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 11:14 'InterProcessCommunication (IPC)'
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

8. ls -lh

- List all files in long format, and show file sizes in a human-readable way

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ls -lh
total 108K
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:29 BankersAlgorithm
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 10:21 CPU_SchedulingAlgorithms
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:25 Critical_Section_Problem
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:30 DeadlockDetectionAlgorithm
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 11:14 'InterProcessCommunication (IPC)'
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:26 ProducerConsumer_Problems
-rw-rw-rw- 1 codespace root       2.5K Oct  8 06:28 ReadMe.md
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:26 'ReaderWriter_Problem & DiningPhilosopher(Semaphores)'
drwxrwxrwx+ 2 codespace codespace 4.0K Oct 22 09:10 Thread_Programming
-rw-rw-rw- 1 codespace root       46K Oct  8 06:28 a.exe
-rwxrwxrwx 1 codespace codespace 24K Oct 22 10:28 a.out
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

9. mkdir

- Create a new directory

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ mkdir test.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ls
BankersAlgorithm      ReadMe.md
CPU_SchedulingAlgorithms 'ReaderWriter_Problem & DiningPhilosopher(Semaphores)'
Critical_Section_Problem Thread_Programming
DeadlockDetectionAlgorithm a.exe
'InterProcessCommunication (IPC)' a.out
ProducerConsumer_Problems test.cpp
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

10. rmdir

- Remove the directory

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ rmdir test.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ls
BankersAlgorithm      ReadMe.md
CPU_SchedulingAlgorithms 'ReaderWriter_Problem & DiningPhilosopher(Semaphores)'
Critical_Section_Problem Thread_Programming
DeadlockDetectionAlgorithm a.exe
'InterProcessCommunication (IPC)' a.out
ProducerConsumer_Problems
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

Lab Task 2: Process creation and termination (OS)

Aim: Implementing process creation and termination commands What are these commands and what function do they perform?

1. **fork()**: Creates a new process by *duplicating* the calling process. The new process, called the child, inherits the *parent's address space, program counter, and open files* but executes as an *independent entity*.
2. **wait()**: Suspends the execution of the parent process until one of its child processes terminates, ensuring proper synchronization and resource reclamation.
3. **exit()**: Ends a process's execution, returning a *status value* to the parent and triggering the operating system to release its resources and remove it from the process table.
4. **signal()**: Handles *asynchronous events* by sending notifications to a process. Signals inform a process of external events like *interrupts or exceptions*, allowing controlled handling instead of abrupt termination.
5. **exec()**: *Replaces* a process's current program with a *new executable*, preserving its process ID but *overwriting its memory space*, enabling *program replacement* within the same process context.

1. Using fork() to create a simple program

processCreationTermination >  fork.cpp > ...

```
1 // 1. Using fork() to create a simple program
2 #include <iostream>
3 #include <unistd.h>
4 using namespace std;
5
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0)
10         cout << "Child process (PID: " << getpid() << ")" << endl;
11     else
12         cout << "Parent process (PID: " << getpid() << ")" << endl;
13
14     return 0;
15 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/fork.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Parent process (PID: 23065)
Child process (PID: 23066)
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

2. Adding wait() to synchronize process

```
processCreationTermination >  wait.cpp >  main()
1  // Adding wait() to synchronize process
2  √ #include <iostream>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  using namespace std;
6
7  √ int main() {
8      pid_t pid = fork();
9
10     if (pid == 0)
11         cout << "Child running (PID: " << getpid() << ")" << endl;
12     √ else {
13         wait(NULL);
14         cout << "Parent waited for child (PID: " << getpid() << ")" << endl;
15     }
16     return 0;
17 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/wait.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Child running (PID: 27122)
Parent waited for child (PID: 27121)
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```


3. Implement signals

processCreationTermination >  signals.cpp >  handler(int)

```
1  #include <iostream>
2  #include <csignal>
3  #include <unistd.h>
4  using namespace std;
5
6  void handler(int sig) {
7      cout << "Signal caught: " << sig << endl;
8      exit(0); // terminate gracefully
9  }
10
11 int main() {
12     signal(SIGINT, handler);
13     cout << "Waiting for Ctrl+C (SIGINT)..." << endl;
14     while (1)
15         sleep(1);
16     return 0;
17 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/signals.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Waiting for Ctrl+C (SIGINT)...
^CSignal caught: 2
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

4. Implement process termination with exit()

processCreationTermination >  exit.cpp >  main()

```
1  #include <iostream>
2  #include <cstdlib>
3  using namespace std;
4
5  int main() {
6      cout<<"This will be printed but after this line next line will not be printed since it's defined after exit() statement. \n";
7      cout << "Process will exit now." << endl;
8      exit(0);
9      cout << "This will not be printed." << endl;
10 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/exit.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
This will be printed but after this line next line will not be printed since it's defined after exit() statement.
Process will exit now.
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

5. The execl() function executes a program specified by its path and accepts arguments as a variable-length argument list.

processCreationTermination >  execl.cpp >  main()

```
1  #include <iostream>
2  #include <unistd.h>
3  using namespace std;
4
5  int main() {
6      execl("/bin/ls", "ls", "-l", (char *)NULL);
7      cout << "If execl fails, this prints." << endl;
8      return 0;
9  }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
total 108
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:29 BankersAlgorithm
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 10:21 CPU_SchedulingAlgorithms
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:25 Critical_Section_Problem
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:30 DeadlockDetectionAlgorithm
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 11:14 'InterProcessCommunication(IPC)'
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:26 ProducerConsumer_Problems
-rw-rw-rw- 1 codespace root 2528 Oct 8 06:28 ReadMe.md
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:26 'ReaderWriter_Problem_&_ DiningPhilosopher(Semaphores)'
drwxrwxrwx+ 2 codespace codespace 4096 Oct 22 09:10 Thread_Programming
-rw-rw-rw- 1 codespace root 46569 Oct 8 06:28 a.exe
-rwxrwxrwx 1 codespace codespace 16544 Oct 25 13:13 a.out
drwxrwxrwx+ 2 codespace codespace 4096 Oct 25 13:13 processCreationTermination
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

6. df: used to see available disk space in each of the partitions of the system

```
processCreationTermination >  df.cpp >  main()
1  #include <iostream>
2  #include <unistd.h>
3  using namespace std;
4
5  int main() {
6      execl("/bin/df", "df", "-h", (char *)NULL);
7      cout << "df failed." << endl;
8      return 0;
9  }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/df.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Filesystem      Size  Used Avail Use% Mounted on
overlay          32G   13G   18G   42% /
tmpfs            64M    0    64M    0% /dev
shm             64M    0    64M    0% /dev/shm
/dev/root        29G   22G   7.7G   74% /vscode
/dev/loop4       32G   13G   18G   42% /workspaces
/dev/sda1        44G   2.1G   40G    5% /tmp
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

7. du: shows disk usage

```
processCreationTermination >  du.cpp >  main()
1  // 7. du: shows disk usage
2  #include <iostream>
3  #include <unistd.h>
4  using namespace std;
5
6  int main() {
7      execl("/usr/bin/du", "du", "-h", ".", (char *)NULL);
8      cout << "du failed." << endl;
9      return 0;
10 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ processCreationTermination/du.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
8.0K    ../git/logs/refs/heads
12K     ../git/logs/refs/remotes/origin
16K     ../git/logs/refs/remotes
28K     ../git/logs/refs
36K     ../git/logs
4.0K    ../git/refs/tags
8.0K    ../git/refs/heads
12K     ../git/refs/remotes/origin
16K     ../git/refs/remotes
32K     ../git/refs
8.0K    ../git/info
8.0K    ../git/objects/39
8.0K    ../git/objects/7d
60K     ../git/objects/c8
8.0K    ../git/objects/06
8.0K    ../git/objects/05
12K     ../git/objects/c9
8.0K    ../git/objects/76
8.0K    ../git/objects/35
8.0K    ../git/objects/2a
12K     ../git/objects/da
8.0K    ../git/objects/14
8.0K    ../git/objects/81
8.0K    ../git/objects/0d
40K     ../git/objects/pack
8.0K    ../git/objects/b6
8.0K    ../git/objects/77
8.0K    ../git/objects/29
8.0K    ../git/objects/cb
8.0K    ../git/objects/b7
8.0K    ../git/objects/85
40K     ../git/objects/f9
8.0K    ../git/objects/8a
8.0K    ../git/objects/b5
8.0K    ../git/objects/26
8.0K    ../git/objects/54
```

⊗ 0 △ 0 (A) 0

```

12K    ./git/objects/09
8.0K   ./git/objects/e6
8.0K   ./git/objects/65
8.0K   ./git/objects/4e
8.0K   ./git/objects/7f
8.0K   ./git/objects/a2
8.0K   ./git/objects/27
8.0K   ./git/objects/aa
8.0K   ./git/objects/92
840K   ./git/objects
84K    ./git/hooks
8.0K   ./git/lfs/cache/locks/refs/heads/main
12K    ./git/lfs/cache/locks/refs/heads
16K    ./git/lfs/cache/locks/refs
20K    ./git/lfs/cache/locks
24K    ./git/lfs/cache
4.0K   ./git/lfs/tmp
32K    ./git/lfs
1.1M   ./git
4.0K   ./BankersAlgorithm
188K   ./Thread_Programming
32K    ./CPU_SchedulingAlgorithms
84K    ./InterProcessCommunication_(IPC)
32K    ./processCreationTermination
4.0K   ./ReaderWriter_Problem_&_DiningPhilosopher(Semaphores)
4.0K   ./ProducerConsumer_Problems
4.0K   ./Critical_Section_Problem
8.0K   ./vscode
4.0K   ./DeadlockDetectionAlgorithm
1.5M   .
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

Lab Task 3: Thread Programming

(Content From My Digital Notes [Thread → [ReadMe.md](#)])

Threads

-> A **thread library** provides the programmer with an **API for creating and managing threads**.

-> There are two primary ways of implementing a thread library.

-> The **first approach** is to provide a **library entirely in user space with no kernel support**. All code and data structures for the library exist in user space. This means that invoking a function in the library results in a local function call in user space and not a system call.

-> The **second approach** is to implement a **kernel-level library supported directly by the operating system**. In this case, code and data structures for the library exist in kernel space. Invoking a function in the API for the library typically results in a system call to the kernel.

-> **Three main thread libraries** are in use today: **POSIX Pthreads, Windows, and Java**.

-> Pthreads, the threads *extension of the POSIX standard*, may be provided as *either a user-level or a kernel-level library*.

-> The Windows thread library is a *kernel-level library* available on Windows systems.

-> The Java thread API allows threads to be created and managed directly in *Java programs*.

-> However, because in most instances the JVM is running on top of a host operating system, the **Java thread API is generally implemented using a thread library available on the host system**. This means that on **Windows systems, Java threads are typically implemented using the Windows API; UNIX, Linux, and macOS systems** typically use Pthreads. -> For POSIX and Windows threading, any data declared globally — that is, declared outside of any function — are shared among all threads belonging to the same process. Because Java has no equivalent notion of global data, access to shared data must be explicitly arranged between threads.

-> *Two general strategies for creating multiple threads: asynchronous threading and synchronous threading.* -> With asynchronous threading, once the parent creates a child thread, the **parent resumes its execution**, so that the **parent and child execute concurrently and independently** of one another. Because the threads are independent, there is typically **little data sharing** between them. Asynchronous threading is the strategy used in the multithreaded server illustrated in Figure 4.2 and is also commonly used for designing responsive user interfaces. -> Synchronous threading occurs when the **parent thread creates one or more children and then must wait for all of its children to terminate before it resumes**. Here, the *threads created by the parent perform work concurrently, but the parent cannot continue until this work has been completed*. Once each thread has finished its work, it terminates and joins with its parent. Only after all of the children have joined can the parent resume execution. Typically, synchronous threading involves **significant data sharing among threads**. For example, the parent thread may combine the results calculated by its various children.

1. Pthreads

1. POSIX Threads (Pthreads) specification is like the abstraction layer or interface (API) for threading in C and C++.
2. Pthreads refers to the POSIX standard (IEEE 1003.1c) defining an API for thread creation and synchronization. This is a specification for thread behavior, not an implementation.
3. Many UNIX-like systems, such as Linux and macOS, include built-in support for the Pthreads specification. Windows does not support Pthreads by default, but there are some third-party libraries that make it possible to use Pthreads on Windows.
4. All Pthreads programs must include the pthread.h header file.
5. Each thread has a set of attributes, including stack size and scheduling information.
6. Synchronization mechanism: a) MUTEX LOCKS: When a thread wants to access a shared resource, it must first acquire the associated mutex - if the mutex is already locked by another thread, the requesting thread enters a waiting state until the lock becomes available. b) Semaphores (signalling concept)

2. Windows Thread

1. We must include the windows.h header file when using the Windows API.
- 2.

2. Java Threads

1. *All Java programs* comprise **at least a single thread of control** — even a simple Java program consisting of only a main() method runs as a single thread in the JVM.
2. There are **two techniques for explicitly creating threads in a Java program**. 3.(a) One approach is to **create a new class that is derived from the Thread class** and to *override its run() method*. 3.(b) An alternative — and *more commonly used* — technique is to **define a class that implements the Runnable interface**.
3. This interface defines a single abstract method with the signature public void run().
4. The code in the run() method of a class that implements Runnable is what executes in a separate thread.
5. An example is shown below: class Task implements Runnable

```
{  
    public void run() {  
        System.out.println("I am a thread.");  
    }  
}
```



7. Invoking the **start() method** for the new **Thread object** does two things:
- a. It **allocates memory and initializes a new thread in the JVM.**
 - b. It **calls the run() method**, making the **thread eligible to be run by the JVM.** (Note again that we *never call the run() method directly*. Rather, we call the **start() method**, and it calls the **run() method** on our behalf.)
8. Recall that the **parent threads in the Pthreads and Windows libraries** use **pthread join()** and **WaitForSingleObject()** (respectively) to **wait for the summation threads to finish before proceeding.**
9. The **join() method in Java** provides similar functionality. (**Notice** that **join()** can throw an **InterruptedException**, which we choose to ignore.)

```
try {  
    worker.join();  
}  
catch (InterruptedException ie) { }
```

10. Classes implementing this interface must define the **execute() method**, which is passed a **Runnable object**.
11. For Java developers, this means **using the Executor rather than creating a separate Thread object and invoking its start() method.**
12. The Executor is used as follows:

```
Executor service = new Executor;  
service.execute(new Task());
```

13. The **Executor framework** is based on the **producer-consumer model**;
14. The **advantage** of this approach is that it **not only divides thread creation from execution but also provides a mechanism for communication between concurrent tasks.**
15. **Data sharing between threads belonging to the same process** occurs *easily in Windows and Pthreads*, since shared data are simply declared *globally*. As a pure object-oriented language, *Java has no such notion of global data*. We can pass parameters to a class that implements **Runnable**, but **Java threads cannot return results.**
16. To address **this need**, the **java.util.concurrent** package additionally defines the **Callable interface**, which behaves similarly to **Runnable** except that a **result can be returned**. Results returned from **Callable tasks** are known as **Future objects**. A result can be retrieved from the **get() method** defined in the **Future interface**.
17. (The **primary difference between the execute() and submit() methods** is that the **former returns no result**, whereas the **latter returns a result as a Future.**)

-> **Implicit Threading** : One way to address the difficulties and better support the design of concurrent and parallel applications (with hundreds and even thousands of threads looming to run concurrently) is to *transfer the creation and management of threading from application developers to compilers and run-time libraries*. -> This strategy, termed implicit threading, is an increasingly popular trend. The advantage of this approach is that *developers only need to identify parallel tasks*, and the *libraries determine the specific details of thread creation and management*. -> **A multithreaded web server**: In this situation, whenever the server receives a request, it creates a separate thread to service the request. Whereas *creating a separate thread is certainly superior to creating a separate process*, a multithreaded server nonetheless has potential problems. -> The *first issue* concerns the *amount of time required to create the thread*, together with the fact that *the thread will be discarded once it has completed its work*. The *second issue* is more troublesome. If we allow each concurrent request to be serviced in a new thread, we have not placed a *bound on the number of threads concurrently active* in the system. **Unlimited threads could exhaust system resources, such as CPU time or memory**. One *solution* to this problem is to use a **thread pool**. -> **WORKING**: When a server receives a request, rather than creating a thread, it instead submits the request to the thread pool and resumes waiting for additional requests. If there is an available thread in the pool, it is awakened, and the request is serviced immediately. If the pool contains no available thread, the task is queued until one becomes free. Once a thread completes its service, it returns to the pool and awaits more work. **Thread pools work well when the tasks submitted to the pool can be executed asynchronously**. -> Thread pools offer these **BENEFITS**: 1. Servicing a request with an existing thread is often faster than waiting to create a thread. 2. A thread pool limits the number of threads that exist at any one point. This is particularly important on systems that cannot support a large number of concurrent threads. 3. Separating the task to be performed from the mechanics of creating the task allows us to use different strategies for running the task. For example, the task could be scheduled to execute after a time delay or to execute periodically.

-> **Threading Issues** : 1. **The fork() and exec() Sys Calls** - The semantics of the fork() and exec() system calls change in a multithreaded program. - If one thread in a program calls fork(), does the new process duplicate all threads, or is the new process single-threaded? Some UNIX systems have chosen to have two versions of fork(), one that duplicates all threads and another that duplicates only the thread that invoked the fork() system call. - The exec() system call typically works in the same way, that is, if a thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process—including all threads. - Which of the two versions of fork() to use depends on the application. If exec() is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameters to exec() will replace the process. In this instance, duplicating only the calling thread is appropriate. If, however, the separate process does not call exec() after forking, the separate process should duplicate all threads.

2. **Signal Handling** - A signal is used in UNIX systems to notify a process that a particular event has occurred. - 1. A signal is generated by the occurrence of a particular event. 2. The signal is delivered to a process. 3. Once delivered, the signal must be handled. - A signal may be handled by one of two possible handlers: 1. A default signal handler 2. A user-defined signal handler 3. **Thread Cancellation** - Thread cancellation involves terminating a thread before it has completed. - A thread that is to be canceled is often referred to as the **target thread**. - Cancellation of a target thread may occur in two different scenarios: 1. **Asynchronous cancellation**. One thread immediately terminates the target thread. 2. **Deferred cancellation**. The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

-> **Implicit Threading** : One way to address the difficulties and better support the design of concurrent and parallel applications (with hundreds and even thousands of threads looming to run concurrently) is to *transfer the creation and management of threading from application developers to compilers and run-time libraries*. -> This strategy, termed implicit threading, is an increasingly popular trend. The advantage of this approach is that *developers only need to identify parallel tasks*, and the *libraries determine the specific details of thread creation and management*. -> **A multithreaded web server**: In this situation, whenever the server receives a request, it creates a separate thread to service the request. Whereas *creating a separate thread is certainly superior to creating a separate process*, a multithreaded server nonetheless has potential problems. -> The *first issue* concerns the **amount of time required to create the thread**, together with the fact that the **thread will be discarded once it has completed its work**. The *second issue* is more troublesome. If we allow each concurrent request to be serviced in a new thread, we have not placed a **bound on the number of threads concurrently active** in the system. **Unlimited threads could exhaust system resources, such as CPU time or memory**. One *solution* to this problem is to use a **thread pool**. -> **WORKING**: When a server receives a request, rather than creating a thread, it instead submits the request to the thread pool and resumes waiting for additional requests. If there is an available thread in the pool, it is awakened, and the request is serviced immediately. If the pool contains no available thread, the task is queued until one becomes free. Once a thread completes its service, it returns to the pool and awaits more work. **Thread pools work well when the tasks submitted to the pool can be executed asynchronously**. -> Thread pools offer these **BENEFITS**: 1. Servicing a request with an existing thread is often faster than waiting to create a thread. 2. A thread pool limits the number of threads that exist at any one point. This is particularly important on systems that cannot support a large number of concurrent threads. 3. Separating the task to be performed from the mechanics of creating the task allows us to use different strategies for running the task. For example, the task could be scheduled to execute after a time delay or to execute periodically.

-> **Threading Issues** : 1. **The fork() and exec() Sys Calls** - The semantics of the fork() and exec() system calls change in a multithreaded program. - If one thread in a program calls fork(), does the new process duplicate all threads, or is the new process single-threaded? Some UNIX systems have chosen to have two versions of fork(), one that duplicates all threads and another that duplicates only the thread that invoked the fork() system call. - The exec() system call typically works in the same way, that is, if a thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process—including all threads. - Which of the two versions of fork() to use depends on the application. If exec() is called immediately after forking, then duplicating all threads is unnecessary, as the program specified in the parameters to exec() will replace the process. In this instance, duplicating only the calling thread is appropriate. If, however, the separate process does not call exec() after forking, the separate process should duplicate all threads.

2. **Signal Handling** - A signal is used in UNIX systems to notify a process that a particular event has occurred. - 1. A signal is generated by the occurrence of a particular event. 2. The signal is delivered to a process. 3. Once delivered, the signal must be handled. - A signal may be handled by one of two possible handlers: 1. A default signal handler 2. A user-defined signal handler 3. **Thread Cancellation** - Thread cancellation involves terminating a thread before it has completed. - A thread that is to be canceled is often referred to as the **target thread**. - Cancellation of a target thread may occur in two different scenarios: 1. **Asynchronous cancellation**. One thread immediately terminates the target thread. 2. **Deferred cancellation**. The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion.

-> We **conclude the chapter** by exploring how threads are implemented in Windows and Linux systems. - **1. Window Threads** : - A Windows application runs as a separate process, and each process may contain one or more threads. The Windows API for creating threads is covered in Section 4.4.2. Additionally, Windows uses the one-to-one mapping described in Section 4.3.2, where each user-level thread maps to an associated kernel thread. - The general components of a thread include: • A thread ID uniquely identifying the thread • A register set representing the status of the processor • A program counter • A user stack, employed when the thread is running in user mode, and a kernel stack, employed when the thread is running in kernel mode • A private storage area used by various run-time libraries and dynamic link libraries (DLLs). - The register set, stacks, and private storage area are known as the context of the thread. The primary data structures of a thread include: • ETHREAD—executive thread block • KTHREAD—kernel thread block • TEB— thread environment block -

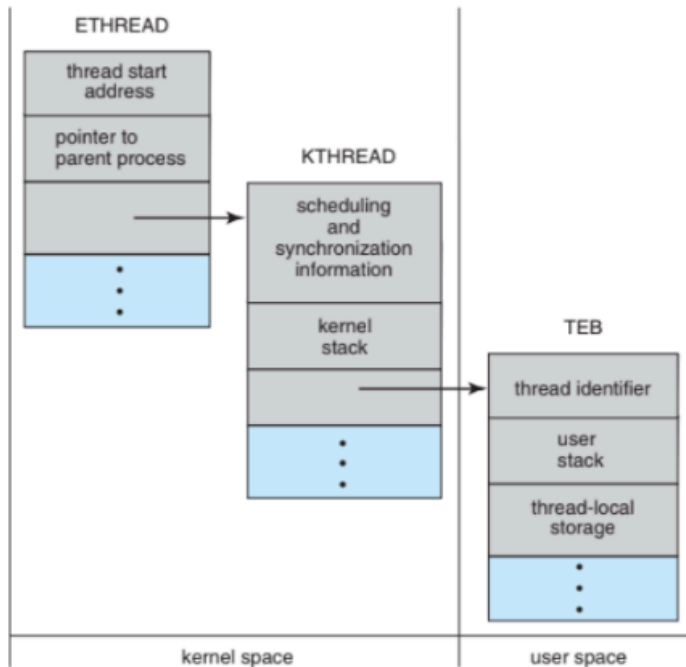


Figure 4.21 Data structures of a Windows thread.

- The KTHREAD includes scheduling and synchronization information for the thread. In addition, the KTHREAD includes the kernel stack (used when the thread is running in kernel mode) and a pointer to the TEB. The ETHREAD and the KTHREAD exist entirely in kernel space; this means that only the kernel can access them. The TEB is a user-space data structure that is accessed when the thread is running in user mode. Among other fields, the TEB contains the thread identifier, a user-mode stack, and an array for thread-local storage. - **2. Linux Threads** : - Linux provides the `fork()` system call with the traditional functionality of duplicating a process, as described in Chapter 3. Linux also provides the ability to create threads using the `clone()` system call. However, Linux does not distinguish between processes and threads. In fact, Linux uses the term *task* — rather than process or thread — when referring to a flow of control within a program. - Using `clone()` in this fashion is equivalent to creating a thread as described in this chapter, since the parent task shares most of its resources with its child task. However, if none of these flags is set when `clone()` is invoked, no sharing takes place, resulting in functionality similar to that provided by the `fork()` system call.

1. Pthread Scheduling

Thread_Programming >  pthread_scheduling_book.cpp > ...

```
1  //This file contained book C code converted to C++ code for p thread execution initially
2  //Then commenting that we wrote a better code. // small addition for understanding threading in action.
3  //final code has all glitches fixed.
4
5  #include <iostream>
6  #include <pthread.h>
7  #define NUM_THREADS 5
8
9  void* runner(void* param);
10
11 int main() {
12     int i, scope;
13     pthread_t tid[NUM_THREADS];
14     pthread_attr_t attr;
15
16     // Initialize thread attributes
17     pthread_attr_init(&attr);
18
19     // Get the current scheduling scope
20     if (pthread_attr_getscope(&attr, &scope) != 0)
21     |     std::cerr << "Unable to get scheduling scope\n";
22     else {
23         if (scope == PTHREAD_SCOPE_PROCESS)
24         |     std::cout << "Current Scope: PTHREAD_SCOPE_PROCESS\n";
25         else if (scope == PTHREAD_SCOPE_SYSTEM)
26         |     std::cout << "Current Scope: PTHREAD_SCOPE_SYSTEM\n";
27         else
28         |     std::cerr << "Illegal scope value.\n";
29     }
30
31     // Set the scheduling scope to SYSTEM
32     pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);
33
34     std::cout << "\nCreating " << NUM_THREADS << " threads...\n";
35
36     // Create threads with unique argument
37     for (i = 0; i < NUM_THREADS; i++) {
38         |     int* threadNum = new int(i); // unique copy of i
39         |     pthread_create(&tid[i], &attr, runner, threadNum);
40     }
41
42     // Wait for all threads to finish
43     for (i = 0; i < NUM_THREADS; i++) {
44         |     pthread_join(tid[i], nullptr);
45     }
46
47     std::cout << "\nAll threads have finished execution.\n";
48     return 0;
49 }
```

Output:

```
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ g++ Thread_Programming/pthread_scheduling_book.cpp
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Current Scope: PTHREAD_SCOPE_SYSTEM

Creating 5 threads...
Thread 0 is running. Thread ID: Thread 2 is running. Thread ID: 131100202256064131100185470656
Thread 3 is running. Thread ID: 131100177077952
Thread 1 is running. Thread ID: 131100193863360

Thread 4 is running. Thread ID: 131100168685248

All threads have finished execution.
○ @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $
```

2. Window Threading

```
Thread_Programming > windowThread_scheduling.cpp > runner(LPVOID)
1  #include <iostream>
2  #include <windows.h>
3
4  #define NUM_THREADS 5
5
6  // Thread function (must match WINAPI signature)
7  DWORD WINAPI runner(LPVOID param) {
8      int threadNum = *(int*)param;
9      std::cout << "Thread " << threadNum
10         << " is running. Thread ID: " << GetCurrentThreadId() << "\n";
11      delete (int*)param;
12      return 0;
13  }
14
15  int main() {
16      HANDLE threads[NUM_THREADS];
17      DWORD threadIds[NUM_THREADS];
18
19      std::cout << "\nCreating " << NUM_THREADS << " threads...\n";
20
21      // Create threads with unique argument
22      for (int i = 0; i < NUM_THREADS; i++) {
23          int* threadNum = new int(i);
24          threads[i] = CreateThread(
25              NULL,           // default security attributes
26              0,              // default stack size
27              runner,          // thread function
28              threadNum,       // thread argument
29              0,              // run immediately
30              &threadIds[i]    // thread identifier
31          );
32          if (threads[i] == NULL) {
33              std::cerr << "Unable to create thread " << i << "\n";
34              delete threadNum;
35          }
36      }
37
38      // Wait for all threads to finish
39      WaitForMultipleObjects(NUM_THREADS, threads, TRUE, INFINITE);
40
41      // Close thread handles
42      for (int i = 0; i < NUM_THREADS; i++) {
43          CloseHandle(threads[i]);
44      }
45
46      std::cout << "\nAll threads have finished execution.\n";
47      return 0;
48  }
```

Output:

```
C:\Users\Srishti\OneDrive\Desktop\Tech NSUT\OS>g++ window_thread_scheduling.cpp

C:\Users\Srishti\OneDrive\Desktop\Tech NSUT\OS>a.exe

Creating 5 threads...
Thread 0 is running. Thread ID: 8420
Thread 2 is running. Thread ID: 12652
Thread 1 is running. Thread ID: 10608
Thread Thread 3 is running. Thread ID: 9632
4 is running. Thread ID: 19124

All threads have finished execution.

C:\Users\Srishti\OneDrive\Desktop\Tech NSUT\OS>
```

3. Java Threading

```
Thread_Programming > J JavaThreads.java > JavaThreads > main(String[])
1 package Thread_Programming;
2 // the code in the book had Integer (which i found, is now, deprecated. thus the code is from perplexity(AI).)
3 import java.util.concurrent.*;
4
5 class Summation implements Callable<Integer> {
6     private int upper;
7
8     public Summation(int upper) {
9         this.upper = upper;
10    }
11
12    // The thread will execute this method
13    @Override
14    public Integer call() {
15        int sum = 0;
16        for (int i = 1; i <= upper; i++) {
17            sum += i;
18        }
19        // Using Integer.valueOf() instead of deprecated new Integer()
20        return Integer.valueOf(sum);
21    }
22 }
23
24 public class JavaThreads {
25     public static void main(String[] args) {
26         if (args.length < 1) {
27             System.out.println("Usage: java ThreadProgramming.JavaThreads <upper_limit>");
28             return;
29         }
30
31         int upper = Integer.parseInt(args[0]);
32
33         // Create a single-thread executor
34         ExecutorService pool = Executors.newSingleThreadExecutor();
35
36         // Submit Summation task
37         Future<Integer> result = pool.submit(new Summation(upper));
38
39         try {
40             // Retrieve and print the result
41             System.out.println("sum = " + result.get());
42         } catch (InterruptedException | ExecutionException ie) {
43             ie.printStackTrace();
44         } finally {
45             // Always shut down the ExecutorService
46             pool.shutdown();
47         }
48     }
49 }
```

0 0 0 0 Java: Lightweight Mode

Output:

```
@SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ g++ Thread_Programming/JavaThreads.java
/usr/bin/ld:Thread_Programming/JavaThreads.java: file format not recognized; treating as linker script
/usr/bin/ld:Thread_Programming/JavaThreads.java:1: syntax error
collect2: error: ld returned 1 exit status
@SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $
```

⊗ 0 △ 0 0 0 Java: Lightweight Mode

Lab Task 4: CPU Scheduling Algorithms

Key Objectives of CPU Scheduling

1. **Maximize CPU Utilization:** Keep the CPU as busy as possible.
2. **Maximize Throughput:** The number of processes completed per unit of time.
3. **Minimize Turnaround Time:** The total time taken for a process to complete.
4. **Minimize Waiting Time:** The total time a process spends waiting in the ready queue.
5. **Minimize Response Time:** The time from when a request is submitted until the first response is produced.

Types of Algorithms:

1. **First Come First Serve (FCFS):** The process that requests the CPU first is allocated the CPU first. The implementation is easily managed by a FIFO queue. When the process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free it is allocated the process at the head of the queue. The running process is then removed from the queue.
2. **Shortest Job First (SJF):** The algorithm associates with each process the length of the process's next CPU burst. When the CPU is available it is assigned to the process with the smallest next CPU burst.
3. **Round Robin (RR):** The algorithm is similar to FCFS, but pre-emption is added to enable the system to switch between processes. A small unit of time called time quantum or time slice is defined. The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for the time interval of up to one quantum.
4. **Pre-emptive Priority Scheduling:** A CPU scheduling algorithm that selects processes for execution based on their priority. This algorithm is preemptive, meaning that the currently running process can be interrupted (or "preempted") if a new process arrives with a higher priority.

1. FCFS

```
CPU_SchedulingAlgorithms > FCFS.cpp > main()
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  //inherently non-preemptive
6  struct Process {
7      int pid;          // Process ID
8      int arrival;      // Arrival time
9      int burst;        // Burst time
10     int start;        // Start time
11     int finish;       // Finish time
12     int waiting;      // Waiting time
13     int turnaround;   // Turnaround time
14 };
15
16 int main() {
17     int n;
18     cout << "Enter the number of processes: ";
19     cin >> n;
20
21     vector<Process> processes(n);
22
23     for (int i = 0; i < n; i++) {
24         processes[i].pid = i + 1;
25         cout << "Enter arrival time for Process " << i + 1 << ": ";
26         cin >> processes[i].arrival;
27         cout << "Enter burst time for Process " << i + 1 << ": ";
28         cin >> processes[i].burst;
29     }
30
31     // Sort processes by arrival time
32     sort(processes.begin(), processes.end(), [](Process a, Process b){
33         return a.arrival < b.arrival;
34     });
35
36     int time_passed = 0;
37     float total_waiting = 0, total_turnaround = 0;
38
39     for (int i = 0; i < n; i++) {
40         if (time_passed < processes[i].arrival) {
41             time_passed = processes[i].arrival; // CPU waits for next process
42         }
43
44         processes[i].start = time_passed;
45         processes[i].finish = processes[i].start + processes[i].burst;
46         processes[i].turnaround = processes[i].finish - processes[i].arrival;
47         processes[i].waiting = processes[i].turnaround - processes[i].burst;
48
49         total_waiting += processes[i].waiting;
50         total_turnaround += processes[i].turnaround;
51
52         time_passed = processes[i].finish;
53     }
54
55     cout << "\nProcess\tArrival\tBurst\tStart\tFinish\tWaiting\tTurnaround\n";
56     for (const auto &p : processes) {
57         cout << p.pid << "\t" << p.arrival << "\t" << p.burst << "\t"
58             << p.start << "\t" << p.finish << "\t" << p.waiting << "\t"
59             << p.turnaround << "\n";
60     }
61
62     cout << "\nAverage Waiting Time: " << total_waiting / n;
63     cout << "\nAverage Turnaround Time: " << total_turnaround / n << endl;
64
65     return 0;
66 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ CPU_SchedulingAlgorithms/FCFS.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Enter the number of processes: 3
Enter arrival time for Process 1: 0
Enter burst time for Process 1: 24
Enter arrival time for Process 2: 0
Enter burst time for Process 2: 3
Enter arrival time for Process 3: 0
Enter burst time for Process 3: 3

Process Arrival Burst   Start   Finish   Waiting Turnaround
1         0        24       0       24       0        24
2         0         3      24       27      24        27
3         0         3      27       30      27        30

Average Waiting Time: 17
Average Turnaround Time: 27
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

2. SJF

CPU_SchedulingAlgorithms >  SJF.cpp > ...

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5  //non-preemptive
6  struct Process {
7      int pid;        // Process ID
8      int arrival;    // Arrival time
9      int burst;      // Burst time
10     int start;      // Start time
11     int finish;     // Finish time
12     int waiting;    // Waiting time
13     int turnaround; // Turnaround time
14     bool completed; // Has process completed?
15 };
16
17 int main() {
18     int n;
19     cout << "Enter the number of processes: ";
20     cin >> n;
21
22     vector<Process> processes(n);
23
24     for (int i = 0; i < n; i++) {
25         processes[i].pid = i + 1;
26         cout << "Enter arrival time for process " << i + 1 << ": ";
27         cin >> processes[i].arrival;
28     }
29     for (int i = 0; i < n; i++) {
30         cout << "Enter burst time for process " << i + 1 << ": ";
31         cin >> processes[i].burst;
32         processes[i].completed = false;
33     }
34
35     int time_passed = 0, completed_count = 0;
36     float total_waiting = 0, total_turnaround = 0;
37
38     // SJF non-preemptive main loop
39     while (completed_count < n) {
40         int idx = -1;
```

```

38 // SJF non-preemptive main loop
39 while (completed_count < n) {
40     int idx = -1;
41     int min_burst = 1e9;
42     // Find the process with minimum burst time among those that have arrived and are not completed
43     for (int i = 0; i < n; i++) {
44         if (!processes[i].completed && processes[i].arrival <= time_passed) {
45             if (processes[i].burst < min_burst) {
46                 min_burst = processes[i].burst;
47                 idx = i;
48             }
49         }
50     }
51     if (idx == -1) {
52         // No process has arrived; move time forward
53         time_passed++;
54         continue;
55     }
56
57     // Assign times
58     processes[idx].start = time_passed;
59     processes[idx].finish = processes[idx].start + processes[idx].burst;
60     processes[idx].turnaround = processes[idx].finish - processes[idx].arrival;
61     processes[idx].waiting = processes[idx].start - processes[idx].arrival;
62     total_waiting += processes[idx].waiting;
63     total_turnaround += processes[idx].turnaround;
64     // Mark as completed
65     processes[idx].completed = true;
66     completed_count++;
67     // Move clock forward
68     time_passed = processes[idx].finish;
69 }
70
71 cout << "\nProcess\tArrival\tBurst\tStart\tFinish\tWaiting\tTurnaround\n";
72 for (const auto &p : processes) {
73     cout << p.pid << "\t" << p.arrival << "\t" << p.burst << "\t"
74         << p.start << "\t" << p.finish << "\t" << p.waiting << "\t"
75         << p.turnaround << "\n";
76 }
77
78 cout << "\nAverage Waiting Time: " << total_waiting / n;
79 cout << "\nAverage Turnaround Time: " << total_turnaround / n << endl;
80
81 return 0;
82 }

```

⊗ 0 △ 0 0 Java: Lightweight Mode

Output:

```

• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ CPU_SchedulingAlgorithms/SJF.cpp
• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Enter the number of processes: 4
Enter arrival time for process 1: 0
Enter arrival time for process 2: 0
Enter arrival time for process 3: 0
Enter arrival time for process 4: 0
Enter burst time for process 1: 6
Enter burst time for process 2: 8
Enter burst time for process 3: 7
Enter burst time for process 4: 3

Process Arrival Burst   Start   Finish   Waiting Turnaround
1         0         6       3       9       3        9
2         0         8      16      24      16       24
3         0         7       9      16       9       16
4         0         3       0       3       0        3

Average Waiting Time: 7
Average Turnaround Time: 13
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

⊗ 0 △ 0 0 Java: Lightweight Mode

3. RoundRobin

CPU_SchedulingAlgorithms > RoundRobin.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5  // Round Robin is inherently Preemptive
6  struct Process {
7      int pid;           // Process ID
8      int arrival;       // Arrival time
9      int burst;         // Burst time (original)
10     int remaining;     // Remaining burst time
11     int finish;        // Completion (exit) time
12     int waiting;       // Waiting time
13     int turnaround;    // Turnaround time
14 };
15
16 int main() {
17     int n, quantum;
18     cout << "Enter the number of processes: ";
19     cin >> n;
20
21     vector<Process> p(n);
22
23     for (int i = 0; i < n; ++i) {
24         p[i].pid = i + 1;
25         cout << "Enter arrival time for process " << i + 1 << ": ";
26         cin >> p[i].arrival;
27     }
28     for (int i = 0; i < n; ++i) {
29         cout << "Enter burst time for process " << i + 1 << ": ";
30         cin >> p[i].burst;
31         p[i].remaining = p[i].burst;
32     }
33     cout << "Enter time quantum: ";
34     cin >> quantum;
35
36     queue<int> ready; // Indices of processes ready to execute
37     int time = 0, completed = 0;
38     vector<bool> queued(n, false);
39
40     // Initial population of ready queue
41     for (int i = 0; i < n; ++i)
42         if (p[i].arrival <= time && !queued[i])
43             ready.push(i), queued[i] = true;
44
45     while (completed < n) {
46         if (ready.empty()) {
```

```

44
45 while (completed < n) {
46     if (ready.empty()) {
47         // Find next earliest arriving process
48         int min_arr = 1e9, idx = -1;
49         for (int i = 0; i < n; ++i)
50             if (!queued[i] && p[i].arrival < min_arr) {
51                 min_arr = p[i].arrival;
52                 idx = i;
53             }
54         time = min_arr;
55         ready.push(idx);
56         queued[idx] = true;
57     }
58     int idx = ready.front();
59     ready.pop();
60
61     int exec_time = min(quantum, p[idx].remaining);
62     // Actual processing
63     time = max(time, p[idx].arrival); // CPU waits if idle
64     p[idx].remaining -= exec_time;
65     time += exec_time;
66
67     // Push newly arrived processes to queue
68     for (int i = 0; i < n; ++i)
69         if (!queued[i] && p[i].arrival <= time) {
70             ready.push(i);
71             queued[i] = true;
72         }
73
74     if (p[idx].remaining == 0) {
75         p[idx].finish = time;
76         ++completed;
77     } else {
78         ready.push(idx); // unfinished, goes to end of queue
79     }
80 }
81 float total_waiting = 0, total_turnaround = 0;
82 cout << "\nProcess\tArrival\tBurst\tFinish\tWaiting\tTurnaround\n";
83 for (int i = 0; i < n; ++i) {
84     p[i].turnaround = p[i].finish - p[i].arrival;
85     p[i].waiting = p[i].turnaround - p[i].burst;
86     total_waiting += p[i].waiting;
87     total_turnaround += p[i].turnaround;
88     cout << p[i].pid << "\t" << p[i].arrival << "\t" << p[i].burst << "\t"
89         << p[i].finish << "\t" << p[i].waiting << "\t" << p[i].turnaround << "\n";
90 }
91 cout << "\nAverage Waiting Time: " << total_waiting / n;
92 cout << "\nAverage Turnaround Time: " << total_turnaround / n << endl;
93 return 0;
94 }

```

Output:

```

• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ CPU_SchedulingAlgorithms/SJF.cpp
• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Enter the number of processes: 4
Enter arrival time for process 1: 0
Enter arrival time for process 2: 0
Enter arrival time for process 3: 0
Enter arrival time for process 4: 0
Enter burst time for process 1: 6
Enter burst time for process 2: 8
Enter burst time for process 3: 7
Enter burst time for process 4: 3

Process Arrival Burst   Start   Finish   Waiting Turnaround
1         0         6       3       9       3        9
2         0         8      16      24      16       24
3         0         7       9      16       9       16
4         0         3       0       3       0        3

Average Waiting Time: 7
Average Turnaround Time: 13
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

⊗ 0 △ 0 0 Java: Lightweight Mode

4. Pre-emptive Priority Scheduling

CPU_SchedulingAlgorithms >  PreemptivePriority.cpp >  main()

```
1 // SJF is the special case of priority scheduling(non-pre-emptive?) when priority is given to the burst time
2 //Priority Scheduling can be intrinsically assigned (system assigned) OR can be extrinsically assigned (user assigned).
3 #include <iostream>
4 #include <vector>
5 using namespace std;
6
7 struct Process {
8     int pid;           // Process ID
9     int arrival;       // Arrival time
10    int burst;          // Burst time (original)
11    int remaining;      // Remaining burst time
12    int priority;       // Priority value (lower value = higher priority)
13    int finish;         // Completion time
14    int waiting;        // Waiting time
15    int turnaround;     // Turnaround time
16    bool completed;     // Completion flag
17 };
18
19 int main() {
20     int n;
21     cout << "Enter the number of processes: ";
22     cin >> n;
23
24     vector<Process> p(n);
25
26     for (int i = 0; i < n; ++i) {
27         p[i].pid = i + 1;
28         cout << "Enter arrival time for process " << i + 1 << ": ";
29         cin >> p[i].arrival;
30     }
31     for (int i = 0; i < n; ++i) {
32         cout << "Enter burst time for process " << i + 1 << ": ";
33         cin >> p[i].burst;
34         p[i].remaining = p[i].burst;
35     }
36     for (int i = 0; i < n; ++i) {
37         cout << "Enter priority for process " << i + 1 << " (lower = higher priority): ";
38         cin >> p[i].priority;
39         p[i].completed = false;
40     }
41
42     int completed = 0, time = 0;
43     float total_waiting = 0, total_turnaround = 0;
44
45     while (completed < n) {
46         int idx = -1;
```

```

44
45     while (completed < n) {
46         int idx = -1;
47         int highest_priority = 1e9;
48         // Find the highest priority process that has arrived and has remaining burst
49         for (int i = 0; i < n; ++i) {
50             if (p[i].arrival <= time && !p[i].completed && p[i].remaining > 0) {
51                 if (p[i].priority < highest_priority) {
52                     highest_priority = p[i].priority;
53                     idx = i;
54                 }
55             }
56         }
57         if (idx == -1) {
58             // No process ready, move time forward
59             time++;
60             continue;
61         }
62         // Preemptively run process with highest priority for 1 time unit
63         p[idx].remaining--;
64         time++;
65         // If the process finishes
66         if (p[idx].remaining == 0) {
67             p[idx].finish = time;
68             p[idx].completed = true;
69             completed++;
70         }
71     }
72
73     // Calculate waiting and turnaround
74     cout << "\nProcess\tArrival\tBurst\tPriority\tFinish\tWaiting\tTurnaround\n";
75     for (int i = 0; i < n; ++i) {
76         p[i].turnaround = p[i].finish - p[i].arrival;
77         p[i].waiting = p[i].turnaround - p[i].burst;
78         total_waiting += p[i].waiting;
79         total_turnaround += p[i].turnaround;
80         cout << p[i].pid << "\t" << p[i].arrival << "\t" << p[i].burst << "\t"
81             << p[i].priority << "\t\t" << p[i].finish << "\t"
82             << p[i].waiting << "\t" << p[i].turnaround << "\n";
83     }
84     cout << "\nAverage Waiting Time: " << total_waiting / n;
85     cout << "\nAverage Turnaround Time: " << total_turnaround / n << endl;
86     return 0;
87 }

```

Output:

```

@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ CPU_SchedulingAlgorithms/PreemptivePriority.cpp
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Enter the number of processes: 5
Enter arrival time for process 1: 0
Enter arrival time for process 2: 0
Enter arrival time for process 3: 0
Enter arrival time for process 4: 0
Enter arrival time for process 5: 0
Enter burst time for process 1: 10
Enter burst time for process 2: 1
Enter burst time for process 3: 2
Enter burst time for process 4: 1
Enter burst time for process 5: 5
Enter priority for process 1 (lower = higher priority): 3
Enter priority for process 2 (lower = higher priority): 1
Enter priority for process 3 (lower = higher priority): 4
Enter priority for process 4 (lower = higher priority): 5
Enter priority for process 5 (lower = higher priority): 2

Process Arrival Burst Priority Finish Waiting Turnaround
1 0 10 3 16 6 16
2 0 1 1 1 0 1
3 0 2 4 18 16 18
4 0 1 5 19 18 19
5 0 5 2 6 1 6

Average Waiting Time: 8.2
Average Turnaround Time: 12
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

Lab Task 5: InterProcess Communication (IPC)

1. Shared Memory

InterProcessCommunication_(IPC) >  producerSharedMemory.cpp >  main()

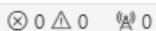
```
1  #include <iostream>
2  #include <fcntl.h>
3  #include <sys/mman.h>
4  #include <sys/stat.h>
5  #include <unistd.h>
6  #include <cstring>
7  #include <sys/wait.h>
8  using namespace std;
9
10 const char* shm_name = "/posix_shm_example";
11 const int SIZE = 4096;
12
13 int main() {
14     int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
15     ftruncate(shm_fd, SIZE);
16
17     void* ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
18
19     pid_t pid = fork();
20
21     if (pid == 0) { // Child process (Reader)
22         sleep(1);
23         cout << "Child reads: " << (char*)ptr << endl;
24         munmap(ptr, SIZE);
25         close(shm_fd);
26         shm_unlink(shm_name);
27     } else { // Parent process (Writer)
28         const char* message = "Hello from parent via POSIX shared memory!";
29         sprintf((char*)ptr, "%s", message);
30         cout << "Parent wrote to shared memory." << endl;
31         wait(NULL);
32     }
33
34     return 0;
35 }
```

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ InterProcessCommunication_(IPC)/producerSharedMemory.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Parent wrote to shared memory.
Child reads: Hello from parent via POSIX shared memory!
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

2. Producer-Consumer using Message Passing (Pipes)

InterProcessCommunication_(IPC) >  producerConsumerMssgPassing.cpp > ...

```
1  #include <iostream>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  using namespace std;
5
6  void producer(int writefd) {
7      for (int i = 1; i <= 10; i++) {
8          write(writefd, &i, sizeof(int));
9          cout << "Producer sent: " << i << endl;
10         sleep(1);
11     }
12     close(writefd);
13 }
14 void consumer(int readfd) {
15     int item;
16     while (read(readfd, &item, sizeof(int)) > 0) {
17         cout << "Consumer received: " << item << endl;
18         sleep(1);
19     }
20     close(readfd);
21 }
22
23 int main() {
24     int pipefd[2];
25     pipe(pipefd);
26
27     if (fork() == 0) {
28         close(pipefd[0]);
29         producer(pipefd[1]);
30         exit(0);
31     }
32
33     if (fork() == 0) {
34         close(pipefd[1]);
35         consumer(pipefd[0]);
36         exit(0);
37     }
38
39     close(pipefd[0]);
40     close(pipefd[1]);
41
42     wait(NULL);
43     wait(NULL);
44
45     return 0;
46 }
```

 0 0 0 0

Output:

```
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ g++ InterProcessCommunication_(IPC)/producerConsumerMsggPassing.cpp
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Producer sent: 1
Consumer received: 1
Producer sent: 2
Consumer received: 2
Producer sent: 3
Consumer received: 3
Producer sent: 4
Consumer received: 4
Producer sent: 5
Consumer received: 5
Producer sent: 6
Consumer received: 6
Producer sent: 7
Consumer received: 7
Producer sent: 8
Consumer received: 8
Producer sent: 9
Consumer received: 9
Producer sent: 10
Consumer received: 10
^@SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ ^C
bash: :s^C: substitution failed
○ @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ █
```

Lab Task 6: Critical Section Problem

Critical_Section_Problem >  PetersonAlgoThreads.cpp > ...

```
1  // Peterson's Algorithm Implementation in C++ (using threads)
2  #include <iostream>
3  #include <thread>
4  #include <atomic>
5
6  std::atomic<bool> flag[2]; // flag for each process
7  std::atomic<int> turn;     // whose turn is it
8  int shared_data = 0;      // critical section resource
9
10 void process(int id) {
11     int other = 1 - id;
12     for (int i = 0; i < 5; ++i) {
13         // Entry section
14         flag[id] = true;
15         turn = other;
16         while (flag[other] && turn == other) {
17             // busy wait (spin)
18         }
19
20         // Critical section
21         std::cout << "Process " << id << " entering critical section." << std::endl;
22         shared_data += 1;
23         std::cout << "Process " << id << " shared_data: " << shared_data << std::endl;
24         std::cout << "Process " << id << " exiting critical section.\n" << std::endl;
25
26         // Exit section
27         flag[id] = false;
28
29         // Remainder section
30         std::this_thread::sleep_for(std::chrono::milliseconds(500));
31     }
32 }
33
34 int main() {
35     flag[0] = flag[1] = false;
36     turn = 0;
37
38     std::thread t1(process, 0);
39     std::thread t2(process, 1);
40
41     t1.join();
42     t2.join();
43
44     std::cout << "Final shared_data: " << shared_data << std::endl;
45
46     return 0;
47 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ Critical_Section_Problem/PetersonAlgoThreads.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Process 0 entering critical section.
Process 0 shared_data: 1
Process 0 exiting critical section.

Process 1 entering critical section.
Process 1 shared_data: 2
Process 1 exiting critical section.

Process 0 entering critical section.
Process 0 shared_data: 3
Process 0 exiting critical section.

Process 1 entering critical section.
Process 1 shared_data: 4
Process 1 exiting critical section.

Process 1 entering critical section.
Process 1 shared_data: 5
Process 1 exiting critical section.

Process 0 entering critical section.
Process 0 shared_data: 6
Process 0 exiting critical section.

Process 1 entering critical section.
Process 1 shared_data: 7
Process 1 exiting critical section.

Process 0 entering critical section.
Process 0 shared_data: 8
Process 0 exiting critical section.

Process 1 entering critical section.
Process 1 shared_data: 9
Process 1 exiting critical section.

Process 0 entering critical section.
Process 0 shared_data: 10
Process 0 exiting critical section.

Final shared_data: 10
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```


Lab Task 7: Producer – Consumer problem using bounded and unbounded buffer.

1. Bounded buffer

ProducerConsumer_Problems > withBoundedBuffer.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  #include <thread>
4  #include <mutex>
5  #include <condition_variable>
6
7  const int BUFFER_SIZE = 5;
8  std::vector<int> buffer;
9  std::mutex mtx;
10 std::condition_variable not_full, not_empty;
11
12 void producer(int items) {
13     for (int i = 0; i < items; ++i) {
14         std::unique_lock<std::mutex> lock(mtx);
15         not_full.wait(lock, []{ return buffer.size() < BUFFER_SIZE; });
16         buffer.push_back(i+1);
17         std::cout << "Produced: " << (i+1) << std::endl;
18         lock.unlock();
19         not_empty.notify_one();
20         std::this_thread::sleep_for(std::chrono::milliseconds(200));
21     }
22 }
23
24 void consumer(int items) {
25     for (int i = 0; i < items; ++i) {
26         std::unique_lock<std::mutex> lock(mtx);
27         not_empty.wait(lock, []{ return !buffer.empty(); });
28         int item = buffer.front();
29         buffer.erase(buffer.begin());
30         std::cout << "Consumed: " << item << std::endl;
31         lock.unlock();
32         not_full.notify_one();
33         std::this_thread::sleep_for(std::chrono::milliseconds(400));
34     }
35 }
36
37 int main() {
38     int N = 10;
39     std::thread t1(producer, N);
40     std::thread t2(consumer, N);
41     t1.join();
42     t2.join();
43     return 0;
44 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ ProducerConsumer_Problems/withBoundedBuffer.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Produced: 6
Consumed: 4
Produced: 7
Produced: 8
Consumed: 5
Produced: 9
Produced: 10
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
```

2. Unbounded buffer

ProducerConsumer_Problems >  withUnboundedBuffer.cpp >  main()

```
1  #include <iostream>
2  #include <thread>
3  #include <queue>
4  #include <mutex>
5  #include <condition_variable>
6
7  std::queue<int> buffer;
8  std::mutex mtx;
9  std::condition_variable not_empty;
10
11 void producer(int items) {
12     for (int i = 1; i <= items; ++i) {
13         {
14             std::lock_guard<std::mutex> lock(mtx);
15             buffer.push(i);
16             std::cout << "Produced: " << i << " (Buffer size: " << buffer.size() << ")" << std::endl;
17         }
18         not_empty.notify_one();
19         std::this_thread::sleep_for(std::chrono::milliseconds(100));
20     }
21 }
22
23 void consumer(int items) {
24     for (int i = 1; i <= items; ++i) {
25         int item = -1;
26         {
27             std::unique_lock<std::mutex> lock(mtx);
28             not_empty.wait(lock, []{ return !buffer.empty(); });
29             item = buffer.front();
30             buffer.pop();
31             std::cout << "Consumed: " << item << " (Buffer size: " << buffer.size() << ")" << std::endl;
32         }
33         std::this_thread::sleep_for(std::chrono::milliseconds(200));
34     }
35 }
36
37 int main() {
38     int N = 15;
39     std::thread prod(producer, N);
40     std::thread cons(consumer, N);
41     prod.join();
42     cons.join();
43     return 0;
44 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ ProducerConsumer_Problems/withUnboundedBuffer.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Produced: 1 (Buffer size: 1)
Consumed: 1 (Buffer size: 0)
Produced: 2 (Buffer size: 1)
Consumed: 2 (Buffer size: 0)
Produced: 3 (Buffer size: 1)
Produced: 4 (Buffer size: 2)
Consumed: 3 (Buffer size: 1)
Produced: 5 (Buffer size: 2)
Produced: 6 (Buffer size: 3)
Consumed: 4 (Buffer size: 2)
Produced: 7 (Buffer size: 3)
Produced: 8 (Buffer size: 4)
Consumed: 5 (Buffer size: 3)
Produced: 9 (Buffer size: 4)
Produced: 10 (Buffer size: 5)
Consumed: 6 (Buffer size: 4)
Produced: 11 (Buffer size: 5)
Produced: 12 (Buffer size: 6)
Consumed: 7 (Buffer size: 5)
Produced: 13 (Buffer size: 6)
Produced: 14 (Buffer size: 7)
Consumed: 8 (Buffer size: 6)
Produced: 15 (Buffer size: 7)
Consumed: 9 (Buffer size: 6)
Consumed: 10 (Buffer size: 5)
Consumed: 11 (Buffer size: 4)
Consumed: 12 (Buffer size: 3)
Consumed: 13 (Buffer size: 2)
Consumed: 14 (Buffer size: 1)
Consumed: 15 (Buffer size: 0)
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

Lab Task 8: Reader Writers problem, Dining Philosophers problem using semaphores.

1. Reader's Writer Problem

ReaderWriter_Problem_&_ DiningPhilosopher(Semaphores) > ReaderWriterPOSIXsemaphores.cpp > main()

```
1  // This solution uses the "writer-priority" pattern for fairness.
2  #include <iostream>
3  #include <thread>
4  #include <vector>
5  #include <chrono>
6  #include <semaphore.h>
7
8  sem_t mutex, wrt;
9  int readcount = 0;
10
11 void reader(int id) {
12     for (int j = 0; j < 3; ++j) {
13         sem_wait(&mutex);
14         ++readcount;
15         if (readcount == 1)
16             sem_wait(&wrt); // first reader blocks writers
17         sem_post(&mutex);
18
19         // CRITICAL SECTION
20         std::cout << "Reader " << id << " is reading.\n";
21         std::this_thread::sleep_for(std::chrono::milliseconds(100));
22
23         sem_wait(&mutex);
24         --readcount;
25         if (readcount == 0)
26             sem_post(&wrt); // last reader unblocks writers
27         sem_post(&mutex);
28
29         std::this_thread::sleep_for(std::chrono::milliseconds(50)); // remainder section
30     }
31 }
32
33 void writer(int id) {
34     for (int j = 0; j < 2; ++j) {
35         sem_wait(&wrt);
36         // CRITICAL SECTION
37         std::cout << "Writer " << id << " is writing.\n";
38         std::this_thread::sleep_for(std::chrono::milliseconds(150));
39         sem_post(&wrt);
40         std::this_thread::sleep_for(std::chrono::milliseconds(100)); // remainder
```

```

40         std::this_thread::sleep_for(std::chrono::milliseconds(100)); // remainder
41     }
42 }
43
44 int main() {
45     sem_init(&mutex, 0, 1);
46     sem_init(&wrt, 0, 1);
47
48     std::vector<std::thread> readers, writers;
49     for (int i = 1; i <= 4; ++i)
50         readers.emplace_back(reader, i);
51     for (int i = 1; i <= 2; ++i)
52         writers.emplace_back(writer, i);
53
54     for (auto& r : readers) r.join();
55     for (auto& w : writers) w.join();
56
57     sem_destroy(&mutex);
58     sem_destroy(&wrt);
59     return 0;
60 }

```

Output:

```

● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ ReaderWriter_Problem_&\ DiningPhilosopher\Semaphores\ReaderWriterPOSIXsemaphores.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Reader 1 is reading.
Reader 2 is reading.
Reader 4 is reading.
Reader 3 is reading.
Writer 1 is writing.
Writer 2 is writing.
Reader 1 is reading.
Reader 2 is reading.
Reader 4 is reading.
Reader 3 is reading.
Writer 1 is writing.
Writer 2 is writing.
Reader 1 is reading.
Reader 2 is reading.
Reader 4 is reading.
Reader 3 is reading.
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

2. Dining Philosopher (Semaphores)

ReaderWriter_Problem_&_DiningPhilosopher(Semaphores) >  DiningPhilosopherPOSIXsemaphores.cpp >  main()

```
1  #include <iostream>
2  #include <thread>
3  #include <vector>
4  #include <semaphore.h>
5  #include <chrono>
6
7  #define N 5
8
9  sem_t forks[N];
10 sem_t mutex;
11
12 void philosopher(int id) {
13     for (int i = 0; i < 3; ++i) {
14         // Thinking section
15         std::cout << "Philosopher " << id << " is thinking.\n";
16         std::this_thread::sleep_for(std::chrono::milliseconds(100));
17
18         // Pickup forks
19         sem_wait(&forks[id]);
20         sem_wait(&forks[(id+1)%N]);
21
22         // Critical section (eating)
23         sem_wait(&mutex);
24         std::cout << "Philosopher " << id << " is eating.\n";
25         sem_post(&mutex);
26         std::this_thread::sleep_for(std::chrono::milliseconds(150));
27
28         // Put down forks
29         sem_post(&forks[id]);
30         sem_post(&forks[(id+1)%N]);
31     }
32 }
33
34 int main() {
35     sem_init(&mutex, 0, 1);
36     for (int i = 0; i < N; ++i)
37         sem_init(&forks[i], 0, 1);
38
39     std::vector<std::thread> philosophers;
40     for (int i = 0; i < N; ++i)
41         philosophers.emplace_back(philosopher, i);
42
43     for (auto& ph : philosophers) ph.join();
44     for (int i = 0; i < N; ++i)
45         sem_destroy(&forks[i]);
46     sem_destroy(&mutex);
47     return 0;
48 }
```

Output:

```
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ g++ ReaderWriter_Problem\_ \_ DiningPhilosopher\Semaphores\)/DiningPhilosopherPOSIXsemaphores.cpp
● @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 1 is thinking.
Philosopher 0 is eating.
Philosopher 3 is thinking.
Philosopher 2 is eating.
Philosopher 2 is thinking.
Philosopher 0 is thinking.
Philosopher 4 is eating.
Philosopher 1 is eating.
Philosopher 4 is thinking.
Philosopher 3 is eating.
Philosopher 1 is thinking.
Philosopher 0 is eating.
Philosopher 3 is thinking.
Philosopher 2 is eating.
Philosopher 0 is thinking.
Philosopher 4 is eating.
Philosopher 2 is thinking.
Philosopher 1 is eating.
Philosopher 3 is eating.
Philosopher 4 is thinking.
Philosopher 0 is eating.
Philosopher 2 is eating.
Philosopher 4 is eating.
○ @SRISHTInsut → /workspaces/OperatingSystem_Sem3 (main) $
```


Lab Task 9: Banker's algorithm.

BankersAlgorithm >  bankersAlgo.cpp >  main()

```
1  #include <iostream>
2  using namespace std;
3
4  const int N_PROCESSES = 5; // Number of processes
5  const int N_RESOURCES = 3; // Number of resources
6
7  int finish[N_PROCESSES];
8  int need[N_PROCESSES][N_RESOURCES];
9  int maxm[N_PROCESSES][N_RESOURCES] = {
10     {7, 5, 3},
11     {3, 2, 2},
12     {9, 0, 2},
13     {2, 2, 2},
14     {4, 3, 3}
15 };
16 int allot[N_PROCESSES][N_RESOURCES] = {
17     {0, 1, 0},
18     {2, 0, 0},
19     {3, 0, 2},
20     {2, 1, 1},
21     {0, 0, 2}
22 };
23 int avail[N_RESOURCES] = {3, 3, 2};
24
25 void calculateNeed() {
26     for (int i = 0; i < N_PROCESSES; i++)
27         for (int j = 0; j < N_RESOURCES; j++)
28             need[i][j] = maxm[i][j] - allot[i][j];
29 }
30
31 bool isSafe() {
32     int work[N_RESOURCES];
33     for (int i = 0; i < N_RESOURCES; i++)
34         work[i] = avail[i];
35     for (int i = 0; i < N_PROCESSES; i++)
36         finish[i] = 0; // false
37
38     int safeSeq[N_PROCESSES], count = 0;
39     while (count < N_PROCESSES) {
40         bool found = false;
41         for (int p = 0; p < N_PROCESSES; p++) {
42             if (finish[p] == 0) {
43                 int j;
44                 for (j = 0; j < N_RESOURCES; j++)
45                     if (need[p][j] > work[j])
46                         break;
```

```

38     int safeSeq[N_PROCESSES], count = 0;
39     while (count < N_PROCESSES) {
40         bool found = false;
41         for (int p = 0; p < N_PROCESSES; p++) {
42             if (finish[p] == 0) {
43                 int j;
44                 for (j = 0; j < N_RESOURCES; j++)
45                     if (need[p][j] > work[j])
46                         break;
47                 if (j == N_RESOURCES) { // If all needs are less than work
48                     for (int k = 0; k < N_RESOURCES; k++)
49                         work[k] += allot[p][k];
50                     safeSeq[count++] = p;
51                     finish[p] = 1; // true
52                     found = true;
53                 }
54             }
55         }
56         if (!found) {
57             cout << "System is not in safe state." << endl;
58             return false;
59         }
60     }
61
62     cout << "System is in safe state.\nSafe sequence is: ";
63     for (int i = 0; i < N_PROCESSES; i++)
64         cout << safeSeq[i] << " ";
65     cout << endl;
66     return true;
67 }
68
69 int main() {
70     calculateNeed();
71     isSafe();
72     return 0;
73 }

```

Output:

```

● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ BankersAlgorithm/bankersAlgo.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
System is in safe state.
Safe sequence is: 1 3 4 0 2
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

Lab Task 10: Deadlock detection algorithm

DeadlockDetectionAlgorithm >  deadlockDetection.cpp >  main()

```
1  #include <iostream>
2  using namespace std;
3
4  // System parameters
5  const int n = 5; // number of processes (threads)
6  const int m = 3; // number of resource types
7
8  int Allocation[n][m] = {
9      {0, 1, 0},
10     {2, 0, 0},
11     {3, 0, 3},
12     {2, 1, 1},
13     {0, 0, 2}
14 };
15
16 int Request[n][m] = { // "Need" matrix: what each process still wants
17     {0, 0, 0},
18     {2, 0, 2},
19     {0, 0, 0},
20     {1, 0, 0},
21     {0, 0, 2}
22 };
23
24 int Available[m] = {0, 0, 0}; // resources available of each type
25
26 void deadlockDetection() {
27     bool Finish[n];
28     int Work[m];
29     for (int i = 0; i < m; ++i) Work[i] = Available[i];
30     for (int i = 0; i < n; ++i) {
31         bool isZero = true;
32         for (int j = 0; j < m; ++j)
33             if (Allocation[i][j] != 0) isZero = false;
34         Finish[i] = isZero;
35     }
36
37     bool progress = true;
38     while (progress) {
39         progress = false;
40         for (int i = 0; i < n; ++i) {
```

```

37     bool progress = true;
38     while (progress) {
39         progress = false;
40         for (int i = 0; i < n; ++i) {
41             if (!Finish[i]) {
42                 bool canProceed = true;
43                 for (int j = 0; j < m; ++j) {
44                     if (Request[i][j] > Work[j]) { // cannot satisfy
45                         canProceed = false;
46                         break;
47                     }
48                 }
49                 if (canProceed) {
50                     // "Complete" this process: release allocation
51                     for (int k = 0; k < m; ++k)
52                         Work[k] += Allocation[i][k];
53                     Finish[i] = true;
54                     progress = true;
55                 }
56             }
57         }
58     }
59
60     // Print deadlocked threads
61     bool deadlock = false;
62     for (int i = 0; i < n; ++i) {
63         if (!Finish[i]) {
64             deadlock = true;
65             cout << "Thread T" << i << " is deadlocked." << endl;
66         }
67     }
68     if (!deadlock)
69         cout << "No deadlock detected. System is safe." << endl;
70 }
71
72 int main() {
73     deadlockDetection();
74     return 0;
75 }

```

Output:

```

● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DeadlockDetectionAlgorithm/deadlockDetection.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
No deadlock detected. System is safe.
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

Lab Task 11: Page replacement algorithms: LRU, LRU-Approximation, FIFO, Optimal.

1. FIFO

```
Page_Replacement >  FIFO_pageReplacement.cpp >  main()
1  #include <iostream>
2  #include <queue>
3  #include <unordered_set>
4  using namespace std;
5
6  int main() {
7      vector<int> pages = {1, 2, 3, 2, 4, 1, 5, 2};
8      int capacity = 3, faults = 0;
9      queue<int> q;
10     unordered_set<int> s;
11
12     for (int p : pages) {
13         if (s.find(p) == s.end()) {
14             if (q.size() == capacity) {
15                 s.erase(q.front());
16                 q.pop();
17             }
18             q.push(p);
19             s.insert(p);
20             faults++;
21         }
22     }
23     cout << "FIFO Page Faults: " << faults << endl;
24     return 0;
25 }
```

Output:

```
• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ Page_Replacement/FIFO_pageReplacement.cpp
• @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
FIFO Page Faults: 7
```

2. LRU

Page_Replacement >  LRU_leastRecentlyUsed.cpp >  main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> pages = {1, 2, 3, 2, 4, 1, 5, 2};
8      int capacity = 3, faults = 0;
9      vector<int> frame;
10
11     for (int p : pages) {
12         auto it = find(frame.begin(), frame.end(), p);
13         if (it == frame.end()) {
14             if (frame.size() == capacity)
15                 frame.erase(frame.begin());
16             faults++;
17         } else {
18             frame.erase(it);
19         }
20         frame.push_back(p);
21     }
22     cout << "LRU Page Faults: " << faults << endl;
23     return 0;
24 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ Page_Replacement/LRU_leastRecentlyUsed.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
LRU Page Faults: 7
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

3. LRU Approximation

Page_Replacement > LRU_approximation.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> pages = {1, 2, 3, 2, 4, 1, 5, 2};
7      int capacity = 3, faults = 0, pointer = 0;
8      vector<int> frame(capacity, -1);
9      vector<bool> ref(capacity, false);
10
11     for (int p : pages) {
12         bool hit = false;
13         for (int i = 0; i < capacity; i++) {
14             if (frame[i] == p) {
15                 ref[i] = true;
16                 hit = true;
17                 break;
18             }
19         }
20         if (!hit) {
21             while (ref[pointer]) {
22                 ref[pointer] = false;
23                 pointer = (pointer + 1) % capacity;
24             }
25             frame[pointer] = p;
26             ref[pointer] = true;
27             pointer = (pointer + 1) % capacity;
28             faults++;
29         }
30     }
31     cout << "LRU-Approx Page Faults: " << faults << endl;
32     return 0;
33 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ Page_Replacement/LRU_approximation.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
LRU-Approx Page Faults: 7
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

4. Optimal

Page_Replacement >  Optimal_pageReplacement.cpp >  main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> pages = {1, 2, 3, 2, 4, 1, 5, 2};
8      int capacity = 3, faults = 0;
9      vector<int> frame;
10
11     for (int i = 0; i < pages.size(); i++) {
12         int p = pages[i];
13         if (find(frame.begin(), frame.end(), p) == frame.end()) {
14             if (frame.size() == capacity) {
15                 int farthest = i + 1, idx = -1;
16                 for (int j = 0; j < frame.size(); j++) {
17                     int k = i + 1;
18                     while (k < pages.size() && pages[k] != frame[j]) k++;
19                     if (k > farthest) {
20                         farthest = k;
21                         idx = j;
22                     }
23                 }
24                 if (idx == -1) frame[0] = p;
25                 else frame[idx] = p;
26             } else {
27                 frame.push_back(p);
28             }
29             faults++;
30         }
31     }
32     cout << "Optimal Page Faults: " << faults << endl;
33     return 0;
34 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ Page_Replacement/Optimal_pageReplacement.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Optimal Page Faults: 5
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```


Lab Task 12: File operation system calls (open, read, close, append etc.)

1. Open, Write and Close

FileOperation >  openWriteClose.cpp >  main()

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main() {
6      ofstream file("example.txt"); // open/create
7      file << "Hello OS Lab!\n";    // write
8      file.close();                 // close
9      cout << "File written and closed.\n";
10     return 0;
11 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ FileOperation/openWriteClose.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
File written and closed.
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

≡ example.txt

```
1  Hello OS Lab!
2
```

2. Read

FileOperation >  read.cpp >  main()

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>
4  using namespace std;
5
6  int main() {
7      ifstream file("example.txt"); // open for reading
8      string line;
9      while (getline(file, line)) {
10         cout << line << endl;
11     }
12     file.close();
13     return 0;
14 }
```

Output:

```

● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ FileOperation/read.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Hello OS Lab!
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

3. Append

FileOperation >  append.cpp >  main()

```

1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4
5  int main() {
6      ofstream file("example.txt", ios::app); // open in append mode
7      file << "Appended line.\n";
8      file.close();
9      cout << "Line appended.\n";
10     return 0;
11 }

```

Output:

```

● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ FileOperation/append.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Line appended.
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $

```

≡ example.txt

```

1  Hello OS Lab!
2  Appended line.
3

```

Lab Task 13: Disk scheduling algorithms: FCFS, SSTF, SCAN, CSCAN, LOOK, CLOOK.

1. FCFS

DiskScheduling > FCFS.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
7      int head = 53, total = 0;
8
9      for (int r : requests) {
10         total += abs(head - r);
11         head = r;
12     }
13
14     cout << "Total FCFS head movement: " << total << endl;
15     return 0;
16 }
```

Output:

```
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/FCFS.cpp
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Total FCFS head movement: 640
@SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

2. SSTF

DiskScheduling > SSTF.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
8      int head = 53, total = 0;
9      vector<bool> visited(requests.size(), false);
10
11     for (int i = 0; i < requests.size(); i++) {
12         int minDist = 1e9, idx = -1;
13         for (int j = 0; j < requests.size(); j++) {
14             if (!visited[j] && abs(head - requests[j]) < minDist) {
15                 minDist = abs(head - requests[j]);
16                 idx = j;
17             }
18         }
19         total += minDist;
20         head = requests[idx];
21         visited[idx] = true;
22     }
23
24     cout << "Total SSTF head movement: " << total << endl;
25     return 0;
26 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/SSTF.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Total SSTF head movement: 236
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

3. SCAN

DiskScheduling >  SCAN_elevatorAlgo.cpp >  main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
8      int head = 53, disk_size = 200, total = 0;
9      sort(requests.begin(), requests.end());
10
11     auto it = lower_bound(requests.begin(), requests.end(), head);
12     for (auto i = it; i != requests.end(); ++i) {
13         total += abs(head - *i);
14         head = *i;
15     }
16     total += abs(head - (disk_size - 1));
17     head = disk_size - 1;
18     for (auto i = it - 1; i >= requests.begin(); --i) {
19         total += abs(head - *i);
20         head = *i;
21     }
22
23     cout << "Total SCAN head movement: " << total << endl;
24     return 0;
25 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/SCAN_elevatorAlgo.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Total SCAN head movement: 331
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

4. CSCAN

DiskScheduling >  CSCAN_circularSCAN.cpp >  main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
8      int head = 53, disk_size = 200, total = 0;
9      sort(requests.begin(), requests.end());
10
11     auto it = lower_bound(requests.begin(), requests.end(), head);
12     for (auto i = it; i != requests.end(); ++i) {
13         total += abs(head - *i);
14         head = *i;
15     }
16     total += abs(head - (disk_size - 1)) + (disk_size - 1);
17     head = 0;
18     for (auto i = requests.begin(); i != it; ++i) {
19         total += abs(head - *i);
20         head = *i;
21     }
22
23     cout << "Total CSCAN head movement: " << total << endl;
24     return 0;
25 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/CSCAN_circularSCAN.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
Total CSCAN head movement: 382
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ █
```

5. LOOK

DiskScheduling >  LOOK.cpp >  main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
8      int head = 53, total = 0;
9      sort(requests.begin(), requests.end());
10
11     auto it = lower_bound(requests.begin(), requests.end(), head);
12     for (auto i = it; i != requests.end(); ++i) {
13         total += abs(head - *i);
14         head = *i;
15     }
16     for (auto i = it - 1; i >= requests.begin(); --i) {
17         total += abs(head - *i);
18         head = *i;
19     }
20
21     cout << "Total LOOK head movement: " << total << endl;
22     return 0;
23 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/LOOK.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
  Total LOOK head movement: 299
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```

6. CLOOK

DiskScheduling > C++ CLOOK.cpp > main()

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  int main() {
7      vector<int> requests = {98, 183, 37, 122, 14, 124, 65, 67};
8      int head = 53, total = 0;
9      sort(requests.begin(), requests.end());
10
11     auto it = lower_bound(requests.begin(), requests.end(), head);
12     for (auto i = it; i != requests.end(); ++i) {
13         total += abs(head - *i);
14         head = *i;
15     }
16     if (it != requests.begin()) {
17         total += abs(head - *requests.begin());
18         head = *requests.begin();
19         for (auto i = requests.begin(); i != it; ++i) {
20             total += abs(head - *i);
21             head = *i;
22         }
23     }
24
25     cout << "Total CLOOK head movement: " << total << endl;
26     return 0;
27 }
```

Output:

```
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ g++ DiskScheduling/CLOOK.cpp
● @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $ ./a.out
  Total CLOOK head movement: 322
○ @SRISHTInsut →/workspaces/OperatingSystem_Sem3 (main) $
```